Nikhil Kale
47

Experiment No. 6

Aim: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Theory:

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a **guard condition** within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a **Voting Smart Contract**, require can be used to check:

- Whether the person calling the function has the right to vote (require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the **chairperson** before granting voting rights.

Thus, require statements enforce **security, correctness, and reliability** in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

- **mapping**:

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

mapping(address => Voter) public voters;

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like **Ballot**, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them **gas efficient** for lookups but limited for enumeration.

- **storage**:

In Solidity, storage refers to the **permanent memory** of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data

stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory**:

In contrast, memory is **temporary storage**, used only for the lifetime of a function

call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must **balance between storage and memory** to ensure efficiency and cost-effectiveness.

3. Why bytes32 Instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in **efficiency and gas optimization**.

- **bytes32** is a **fixed-size type**, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a **dynamically sized type**, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the **Web3 Type Converter** help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

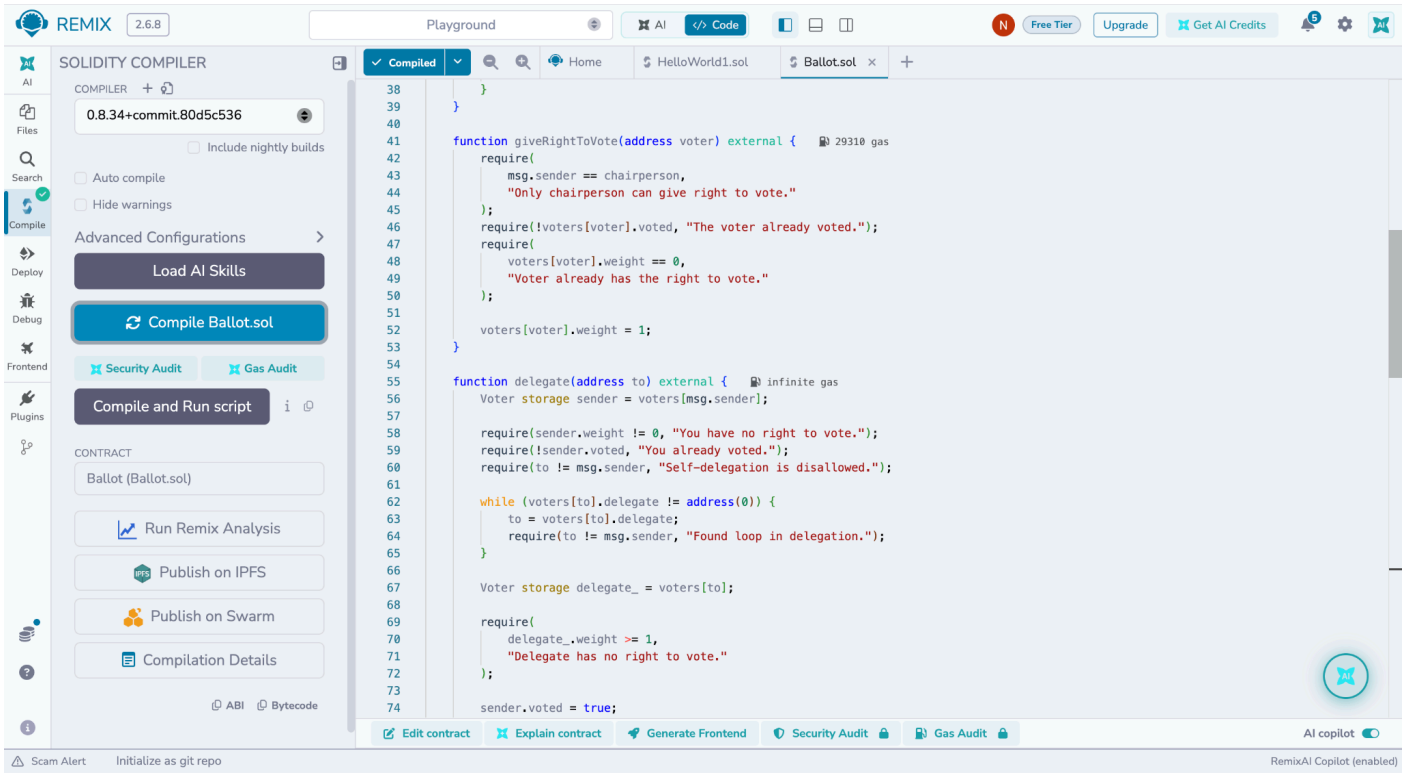
Program -

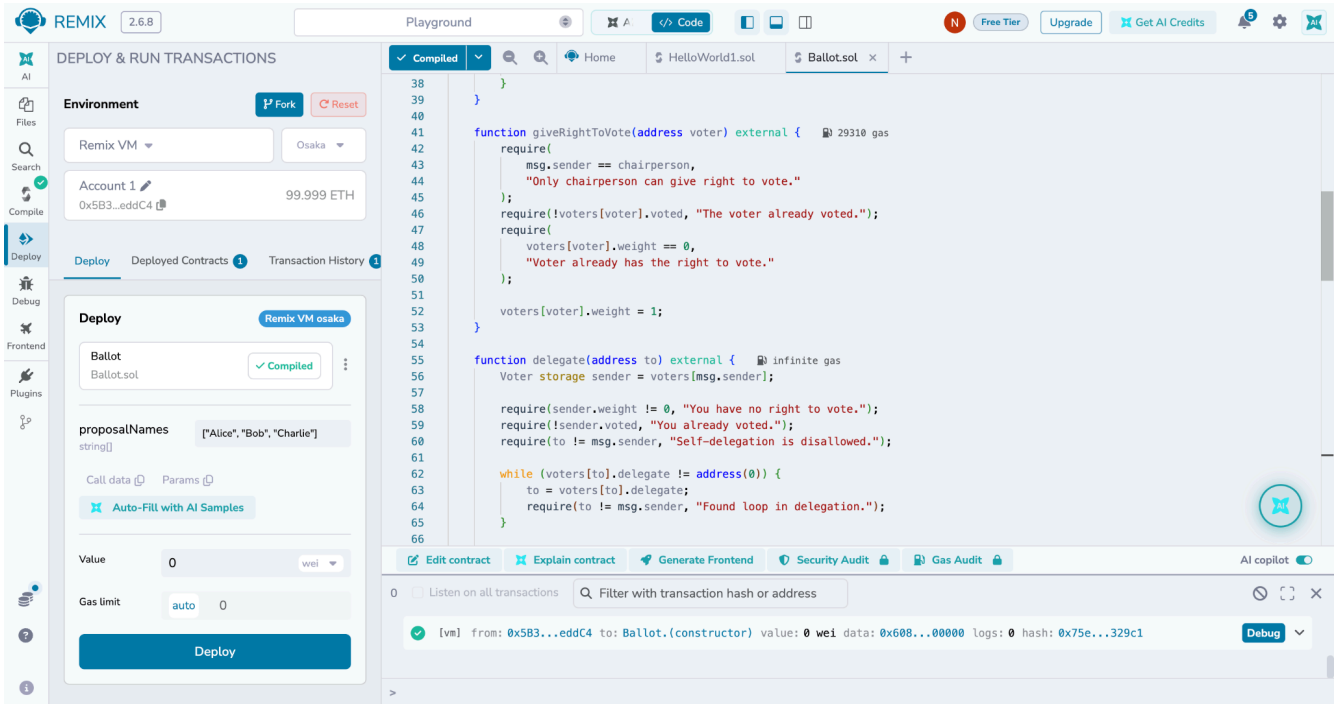
The Solidity program implements the Ballot contract with the following functions:

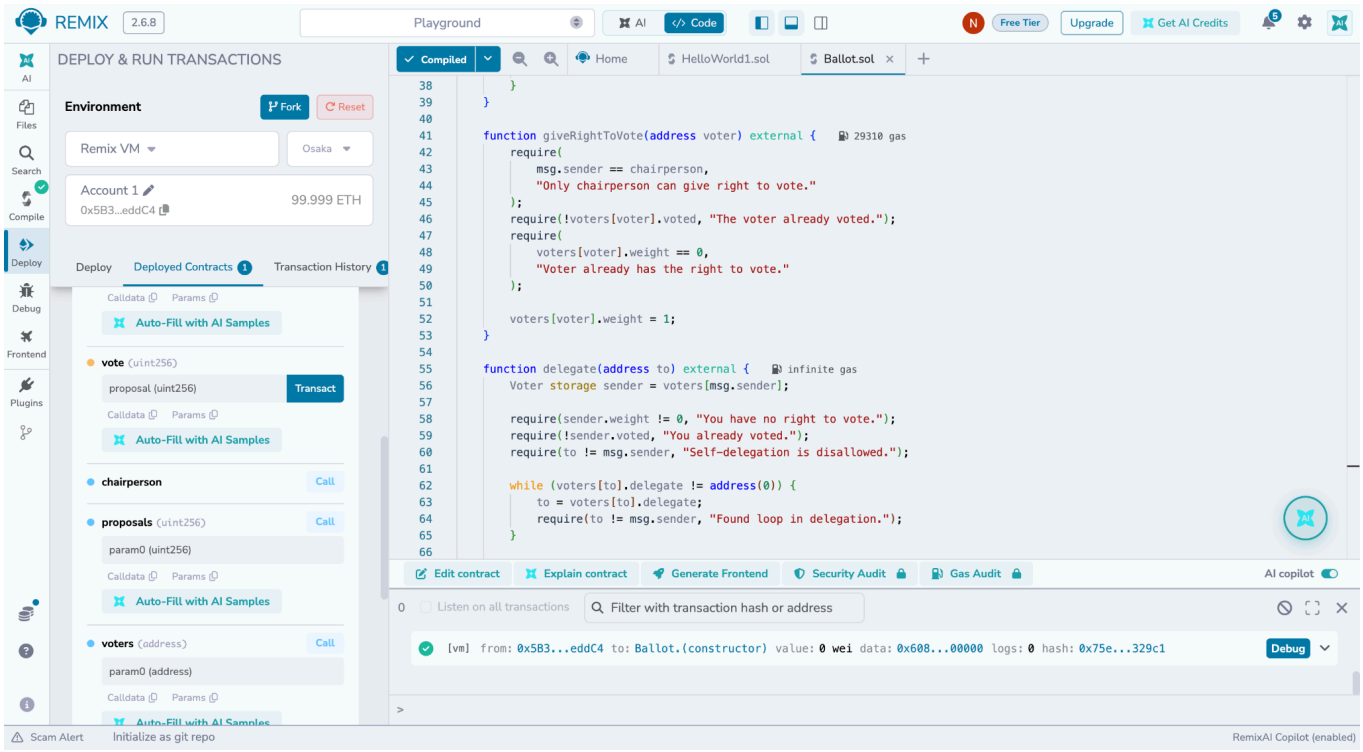
- giveRightToVote(address voter) — grants voting rights to an eligible account.
- delegate(address to) — delegates voting weight to another eligible voter.
- vote(uint proposal) — records a vote for a selected proposal.
- winningProposal() — returns the index of the winning proposal.
- winnerName() — returns the name of the winning proposal.

The complete source code is provided in the Ballot.sol file used during the experiment.

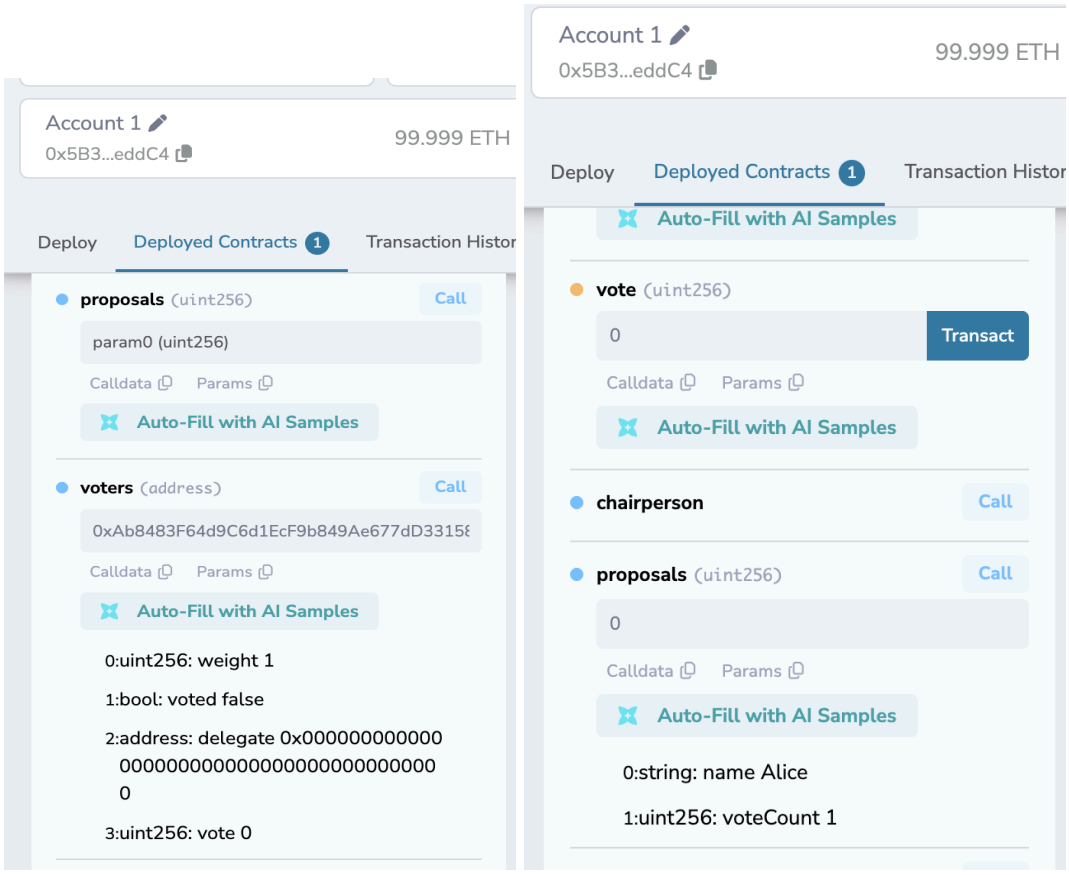
Output -

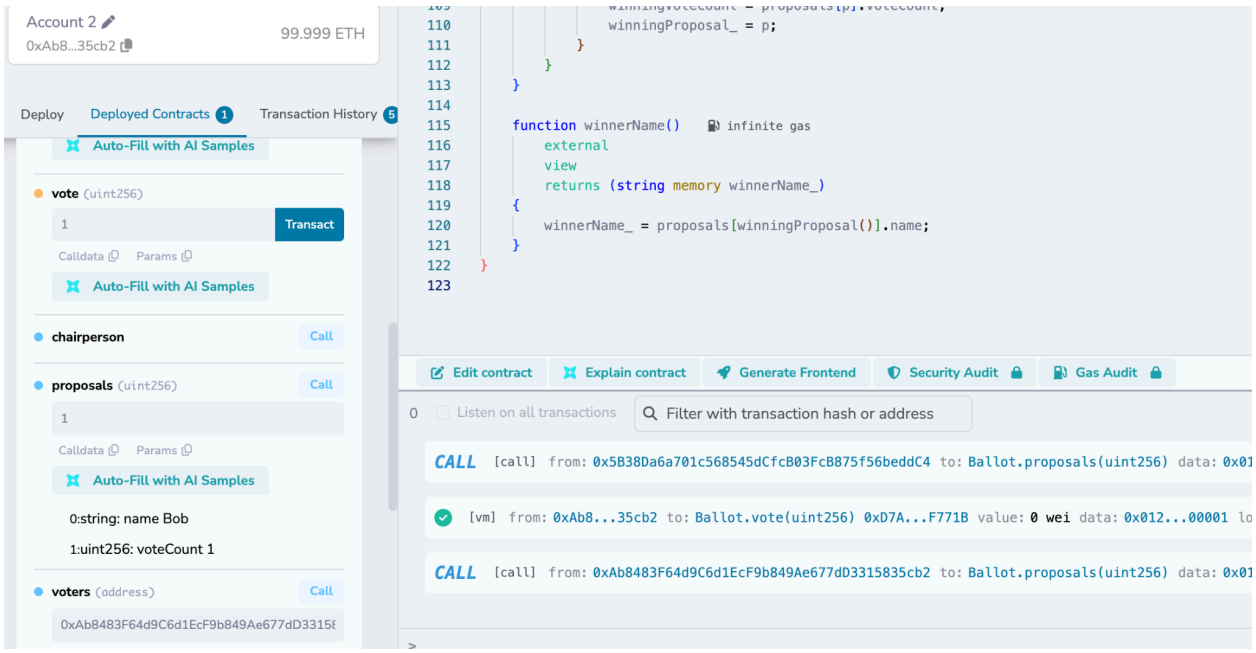

REMIX 2.6.8
Playground
SOLIDITY COMPILER
COMPILER
0.8.34+commit.80d5c536
Include nightly builds
Auto compile
Hide warnings
Advanced Configurations
Load AI Skills
Compile Ballot.sol
Security Audit
Gas Audit
Compile and Run script
CONTRACT
Ballot (Ballot.sol)
Run Remix Analysis
Publish on IPFS
Publish on Swarm
Compilation Details
ABI
Bytecode
Compiled
Home
HelloWorld1.sol
Ballot.sol
function giveRightToVote(address voter) external { 29310 gas
require(
msg.sender == chairperson,
"Only chairperson can give right to vote."
);
require(!voters[voter].voted, "The voter already voted.");
require(
voters[voter].weight == 0,
"Voter already has the right to vote."
);
voters[voter].weight = 1;
}
function delegate(address to) external { infinite gas
Voter storage sender = voters[msg.sender];
require(sender.weight != 0, "You have no right to vote.");
require(!sender.voted, "You already voted.");
require(to != msg.sender, "Self-delegation is disallowed.");
while (voters[to].delegate != address(0)) {
to = voters[to].delegate;
require(to != msg.sender, "Found loop in delegation.");
}
Voter storage delegate_ = voters[to];
require(
delegate_.weight >= 1,
"Delegate has no right to vote."
);
sender.voted = true;
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)

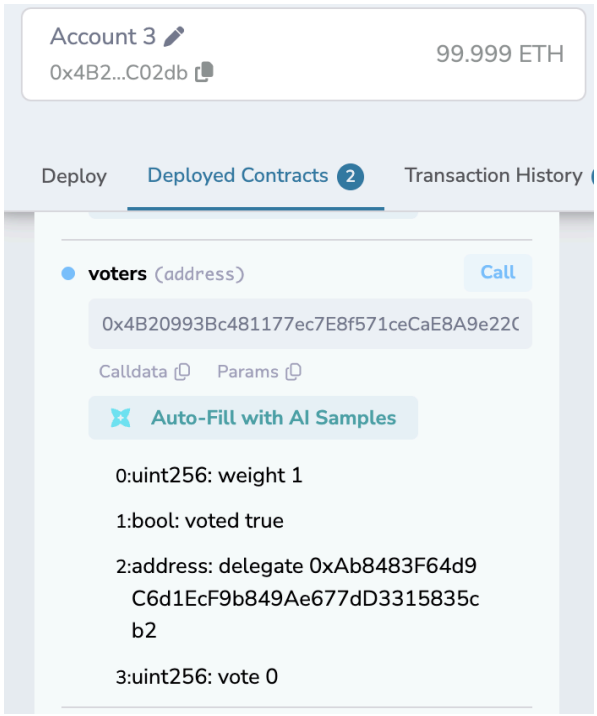

REMIX 2.6.8
Playground
DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deploy
Deployed Contracts 1
Transaction History 1
Deploy
Remix VM osaka
Ballot
Ballot.sol
Compiled
proposalNames
string[]
["Alice", "Bob", "Charlie"]
Call data
Params
Auto-Fill with AI Samples
Value
0
wei
Gas limit
auto
0
Deploy
Compiled
Home
HelloWorld1.sol
Ballot.sol
function giveRightToVote(address voter) external { 29310 gas
require(
msg.sender == chairperson,
"Only chairperson can give right to vote."
);
require(!voters[voter].voted, "The voter already voted.");
require(
voters[voter].weight == 0,
"Voter already has the right to vote."
);
voters[voter].weight = 1;
}
function delegate(address to) external { infinite gas
Voter storage sender = voters[msg.sender];
require(sender.weight != 0, "You have no right to vote.");
require(!sender.voted, "You already voted.");
require(to != msg.sender, "Self-delegation is disallowed.");
while (voters[to].delegate != address(0)) {
to = voters[to].delegate;
require(to != msg.sender, "Found loop in delegation.");
}
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0x75e...329c1
Debug

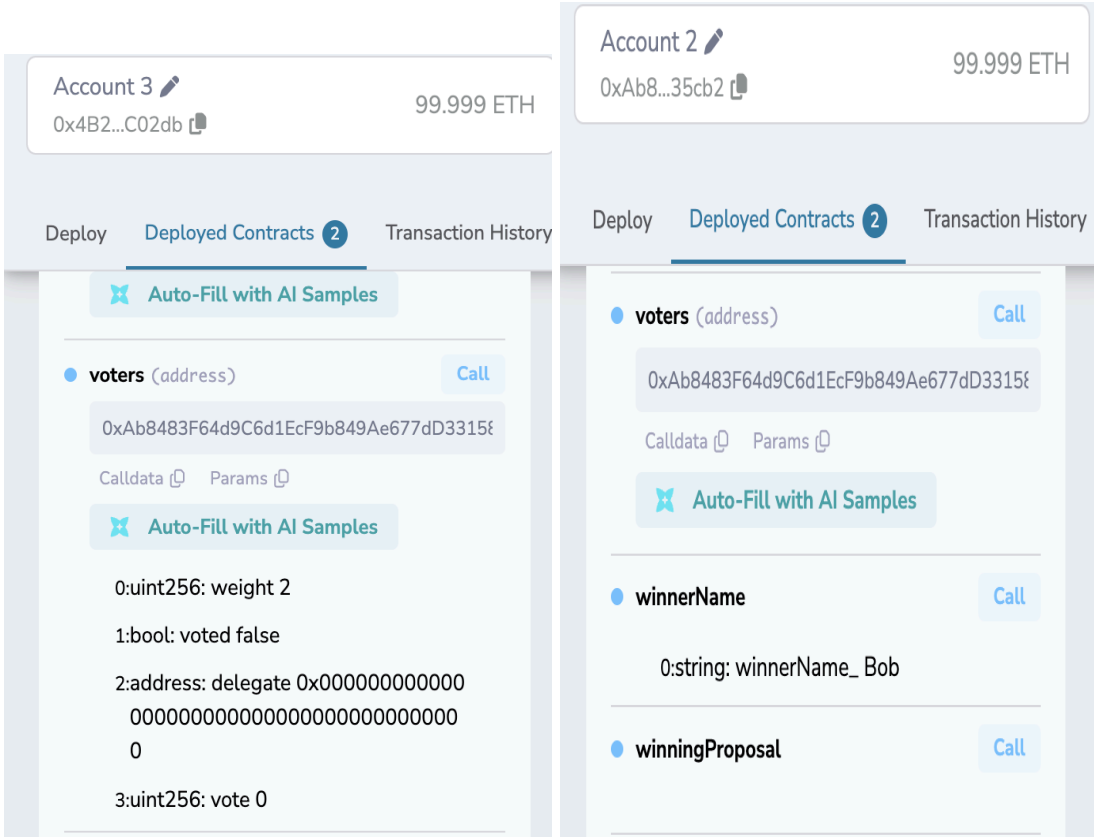

REMIX 2.6.8
Playground
AI
Code
Free Tier
Upgrade
Get AI Credits
DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deploy
Deployed Contracts 1
Transaction History 1
Auto-Fill with AI Samples
vote (uint256)
proposal (uint256)
Transact
chairperson
Call
proposals (uint256)
param0 (uint256)
voters (address)
param0 (address)
Compiled
Home
HelloWorld1.sol
Ballot.sol
function giveRightToVote(address voter) external { 29310 gas
require(msg.sender == chairperson, "Only chairperson can give right to vote.");
require(!voters[voter].voted, "The voter already voted.");
require(voters[voter].weight == 0, "Voter already has the right to vote.");
voters[voter].weight = 1;
function delegate(address to) external { infinite gas
Voter storage sender = voters[msg.sender];
require(sender.weight != 0, "You have no right to vote.");
require(!sender.voted, "You already voted.");
require(to != msg.sender, "Self-delegation is disallowed.");
while (voters[to].delegate != address(0)) {
to = voters[to].delegate;
require(to != msg.sender, "Found loop in delegation.");
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0x75e...329c1
Debug
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)

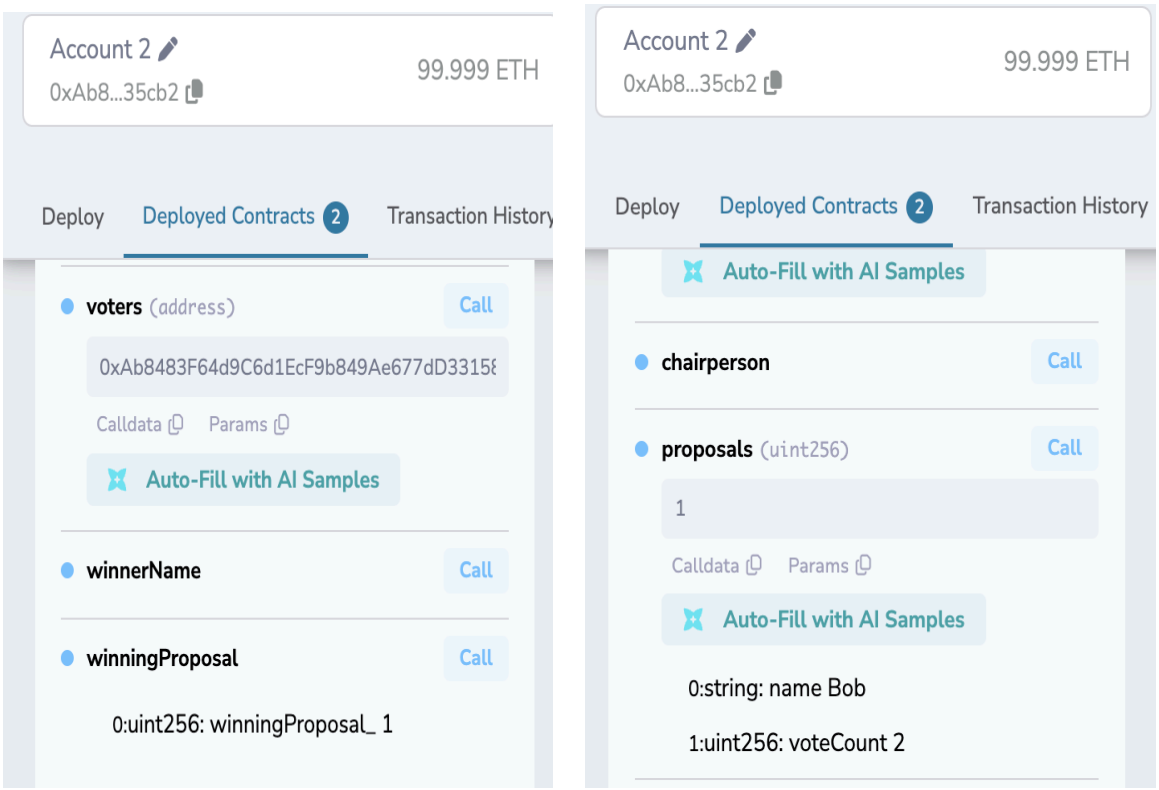

Deploy
Deployed Contracts 1
Transaction History 3
delegate (address)
to (address)
Transact
giveRightToVote (address)
0x4B20993Bc481177ec7E8f571ceC
vote (uint256)
proposal (uint256)
chairperson
Call
function winnerName() infinite gas
external
view
returns (string memory winnerName_)
winnerName_ = proposals[winningProposal()].name;
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
AI copilot
Listen on all transactions
Filter with transaction hash or address
[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xD7A...F771B value: 0 wei data: 0x9e7...35cb2 logs: 0 hash: 0x7b0...4f19b
[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xD7A...F771B value: 0 wei data: 0x9e7...c02db logs: 0 hash: 0x964...9a05c
Debug


Account 1
0x5B3...eddC4
99.999 ETH
Deploy
Deployed Contracts 1
Transaction Histor
proposals (uint256)
Call
param0 (uint256)
Calldata
Params
Auto-Fill with AI Samples
voters (address)
Call
0xAb8483F64d9C6d1EcF9b849Ae677dD3315
Calldata
Params
Auto-Fill with AI Samples
0:uint256: weight 1
1:bool: voted false
2:address: delegate 0x0000000000000000000000000000000000000000
3:uint256: vote 0
Account 1
0x5B3...eddC4
99.999 ETH
Deploy
Deployed Contracts 1
Transaction Histor
Auto-Fill with AI Samples
vote (uint256)
0
Transact
Calldata
Params
Auto-Fill with AI Samples
chairperson
Call
proposals (uint256)
Call
0
Calldata
Params
Auto-Fill with AI Samples
0:string: name Alice
1:uint256: voteCount 1


Account 2
0xAb8...35cb2
99.999 ETH
Deploy
Deployed Contracts 1
Transaction History 5
Auto-Fill with AI Samples
vote (uint256)
1
Transact
Calldata
Params
Auto-Fill with AI Samples
chairperson
Call
proposals (uint256)
Call
1
Calldata
Params
Auto-Fill with AI Samples
0:string: name Bob
1:uint256: voteCount 1
voters (address)
Call
0xAb8483F64d9C6d1EcF9b849Ae677dD3315
winningProposal_ = p;
}
}
}
function winnerName()
infinite gas
external
view
returns (string memory winnerName_)
{
winnerName_ = proposals[winningProposal()].name;
}
}
Edit contract
Explain contract
Generate Frontend
Security Audit
Gas Audit
0
Listen on all transactions
Filter with transaction hash or address
CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Ballot.proposals(uint256) data: 0x01
[vm] from: 0xAb8...35cb2 to: Ballot.vote(uint256) 0xD7A...F771B value: 0 wei data: 0x012...00001 lo
CALL [call] from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2 to: Ballot.proposals(uint256) data: 0x01


Account 3
0x4B2...C02db
99.999 ETH
Deploy
Deployed Contracts 2
Transaction History
voters (address)
Call
0x4B20993Bc481177ec7E8f571ceCaE8A9e22C
Calldata
Params
Auto-Fill with AI Samples
0:uint256: weight 1
1:bool: voted true
2:address: delegate 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
3:uint256: vote 0


Account 3
0x4B2...C02db
99.999 ETH
Deploy
Deployed Contracts 2
Transaction History
Auto-Fill with AI Samples
voters (address)
Call
0xAb8483F64d9C6d1EcF9b849Ae677dD3315
Calldata
Params
Auto-Fill with AI Samples
0:uint256: weight 2
1:bool: voted false
2:address: delegate 0x0000000000000000000000000000000000000000
3:uint256: vote 0
Account 2
0xAb8...35cb2
99.999 ETH
Deploy
Deployed Contracts 2
Transaction History
voters (address)
Call
0xAb8483F64d9C6d1EcF9b849Ae677dD3315
Calldata
Params
Auto-Fill with AI Samples
winnerName
Call
0:string: winnerName_ Bob
winningProposal
Call



The contract compiled and deployed successfully in Remix VM. The chairperson was able to grant voting rights to other accounts. Authorized voters cast votes, and vote delegation increased the delegate's voting weight.

For the demonstration, Account 2 delegated its vote to Account 1, which then voted for Bob. Bob received two votes, while Alice and Charlie received zero votes. The winningProposal() function returned index 1, and winnerName() returned Bob.

Conclusion

In this experiment, a voting smart contract was created, compiled and deployed using Solidity and Remix IDE. The experiment demonstrated voting-right authorization, vote casting, vote delegation, transaction validation and winner determination. It also provided practical understanding of require, mapping, storage, memory, and the differences between bytes32 and string in Solidity.